

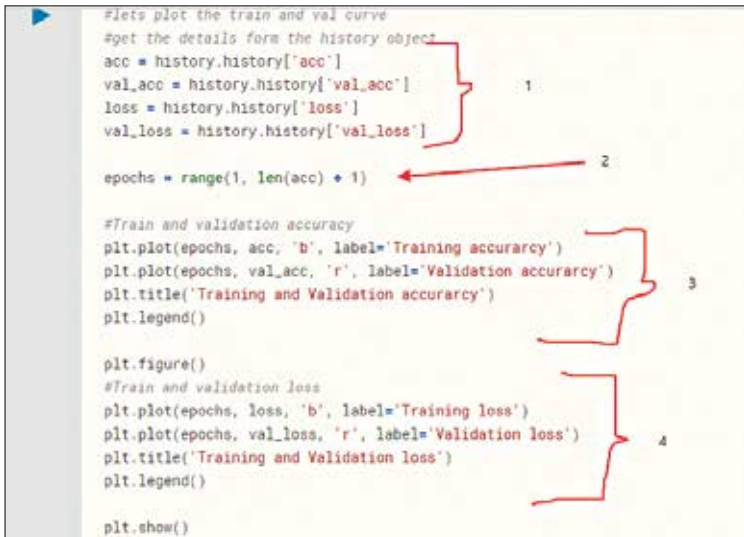
Saving the model:

Next, we save our model, so that we can re-use it anytime instead of training again when we run our notebook again. Both the weights and the model are stored in different.h5 extension file using Keras.

Plotting Various Graphs:

Now, we will plot some graphs of the accuracy and the losses in both the training and validation data sets to gain insights. Follow these steps:

1. Keras always calculates and saves the metric that we specified when we compiled our model in a variable called history. It contains all the updates that happened during training. We can extract these values to plot various accuracy and losses graphs. We get the size of our epoch from the number of values in the 'acc' metric list.
2. Now we plot the loss against the epoch size and the accuracy against the epoch size using matplotlib function. Its code is given in the following image:



```
#lets plot the train and val curve
#get the details form the history object
acc = history.history['acc']
val_acc = history.history['val_acc']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

#Train and validation accuracy
plt.plot(epochs, acc, 'b', label='Training accuracy')
plt.plot(epochs, val_acc, 'r', label='Validation accuracy')
plt.title('Training and Validation accuracy')
plt.legend()

plt.figure()
#Train and validation loss
plt.plot(epochs, loss, 'b', label='Training loss')
plt.plot(epochs, val_loss, 'r', label='Validation loss')
plt.title('Training and Validation loss')
plt.legend()

plt.show()
```

Figure : Code to plot various graphs using the matplotlib plot function

Insights from the plots:

Looking at the plot of accuracy vs epoch, we can infer the following:

1. We are not overfitting the data as the training, and the validation set